

- Bug List (date_1/13)
 - 1、widen2指令下，没有把操作数正确分配到对应加法器上
 - BUG说明
 - 输入输出
 - 定位分析
 - Q1:BF16数据没有正确驱动到对应vfadd0和vfadd1
 - Q2:第一组的vfadd0也并没有完成正常的运算
 - 修改建议
 - 2、widen2指令下，NaN判断没有正确传递
 - BUG说明
 - 定位分析
 - 修改建议
 - 3、比较指令下，func6的控制信号和数据信号没有对齐，导致出现错误(1.8更新)
 - BUG说明：
 - 输入输出：
 - 定位分析
 - 修改建议：
 - 4、vfmv.f.s指令，io_out_rd_valid信号和io_out_valid信号不同时拉高。（1.11更新）
 - BUG说明：
 - 修改建议：
 - 5、多指令序列发生后的保序以及流水问题（1.11 更新）
 - 6、Widen情况下，NaN的判断逻辑有误（对Bug2的增补）
 - BUG说明
 - 定位分析
 - 7、Widen2情况下，内部实现FP16向FP32的转换后，移位标准错误导致精度损失
 - BUG说明：
 - 深入说明
 - 补充Case
 - 修改建议：
 - 8、VFMIN MAX指令，在操作数有来自rs的情况下，选择输出错误。
 - BUG说明：
 - 定位分析
 - 修该建议
- 重测情况
 - 问题清单
 - 1.13 更新
 - 1.12 更新 BUG（1） rerun fail
 - 定位分析：
 - 修改建议
 - 1.12 更新BUG（4）（5），Rerun Pass
- RTL更新情况

Bug List (date_1/13)

更新日期：2026/1/13 17:30

1/12：RTL已经更新为1.11得到的版本，DE反馈对bug1、2、4、5进行了增补，目前对对应case进行了重测，反馈一些问题。

RTL针对按照重测1的情况对进行了增补，Bug1目前成功fix，Bug2仍然没完全修复。具体内容更新在6上面。

1/13：新增Bug7，这个Bug比较corner而且很有价值，修改起来我觉得需要谨慎思考内部微架构，因为通路上会复用多种类型的操作数

1/13：新增Bug8，比较简单的选择逻辑少考虑了一些情况

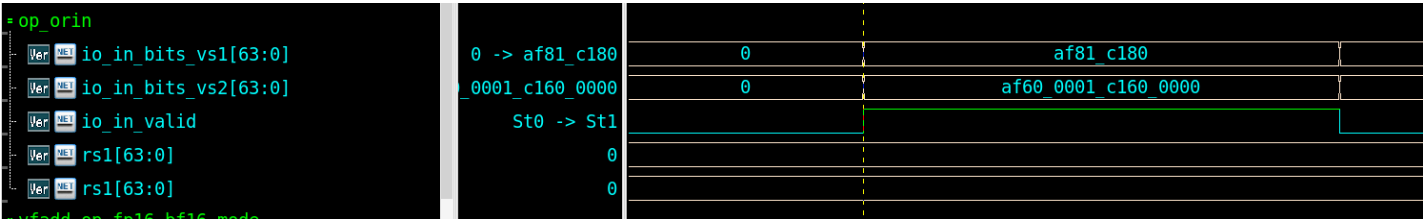
1、widen2指令下，没有把操作数正确分配到对应加法器上

BUG说明

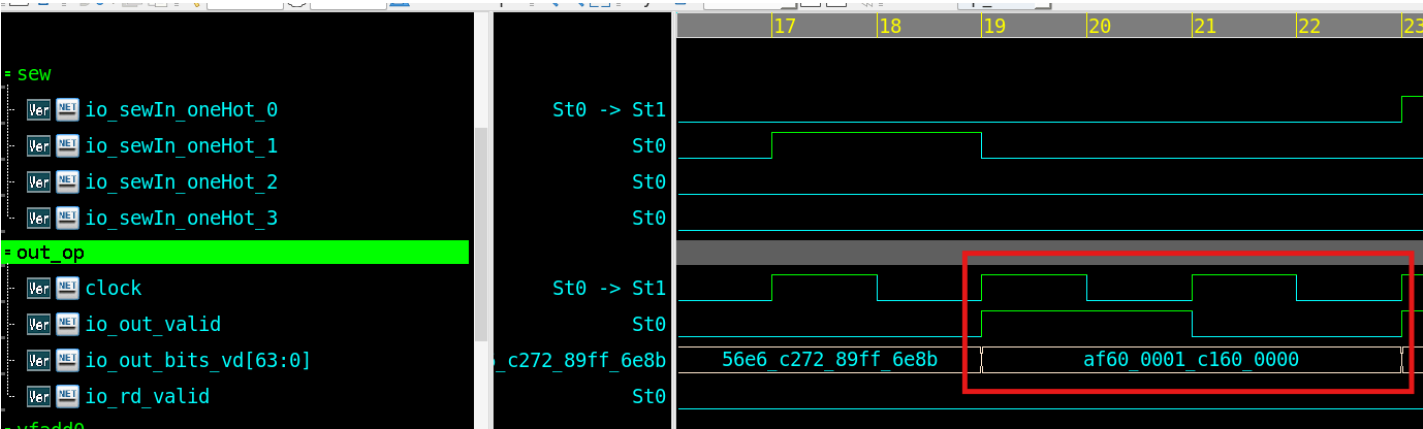
- tc_name: test_tc2_widen_normal_fvv
- tc_seed:0 widen2指令下，输入一组bf6数与一组FP32相加，未得到正确数据
- 波形文件: DUTLaneFAdd_test_tc2_widen_normal_d3cc9d44.fst
- log文件: test_tc2_widen_normal_fvv_0.log

输入输出

输入：



输出：



输入操作数组：

Pair	Operand	Precision	Category	Sign	Exponent	Mantissa	Value Bits	Float Value
0	OPA	BF16	NORMAL	1	0x83	0x0	0x0000c180	-16
0	OPB	FP32	NORMAL	1	0x82	0x600000	0xc1600000	-14.0
1	OPA	BF16	NORMAL	1	0x5f	0x1	0x0000af81	-2.3465e-10
1	OPB	FP32	NORMAL	1	0x5e	0x600001	0xaf600001	-2.0372683e-10

```
[Cycle: 10 Time:21] [INFO] [vfadd_rm] Expected VD: 0xaff10000c1f00000
[Cycle: 10 Time:21] [INFO] [vfadd_rm] Actual   VD: 0xaf600001c1600000
```

第一组-16+-14=-30,dut仍然输出-14.0，即没有正确相加

定位分析

Q1:BF16数据没有正确驱动到对应vfadd0和vfadd1

vfadd0

Ver

io_in_bits_uop_ctrl_widen2

Ver

io_in_bits_uop_ctrl_widen

Ver

io_b[31:0]

Ver

io_a[31:0]

St0 -> St1

St0

0 -> af81_c180

0 -> c160_0000

0

af81_c180

0

c160_0000

vfadd0的io_b被两个bf16驱动

vfadd1

Ver

io_valid_in

Ver

io_b[31:0]

Ver

io_a[31:0]

Ver

is_16

St0 -> St1

St0

0 -> af60_0001

St0 -> St1

0

af60_0001

而vfadd1的io_b则直接没有正确获取到操作数。

Q2:第一组的vfadd0也并没有完成正常的运算

根据RTL代码描述：

vfadd0负责的32Bit被分成了Low16域和High16，相当于两个bf16

```
// @[src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala 139:58]
951 assign fadd_extSig fp32 io b sig = {sig_adjust_subnorm_high_b,3'h0};
// @[src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala 139:58]
```

(LaneFADD.v 第951行)

```

771   sig adjust subnorm_32_0; // @[src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala 132:38]
772   wire [23:0] sig_adjust_subnorm_high_b_T = { sig_adjust_subnorm_16_3_T, frac_high_b_16, 13'h0};
// @[src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala 133:71]
773   wire [23:0] sig_adjust_subnorm_high_b = is_16 ? sig_adjust_subnorm_high_b_T : sig_adjust_subnorm_32_1;
// @[src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala 133:38]

```

第773行代码

这里表明操作数b的小数位在16位模式下是从sub_normal_high这个信号得到的（32位模式下忽略）

根据Spec描述：

uopldx: 对于widen或者widen2指令，针对输入数据为16比特操作数的情况，如果uopldx(0)为0，则输入的低32比特有效（包含两个16比特数据）；如果uopldx(0)为1，则输入的高32比特有效（包含两个16比特数据）

vs1的两个bf16应该自然排列，那么小数位应该取低16位的bf16（即第一个bf16作为操作数）。但是这里取到了高16位

修改建议

DE的此处的意图实际上是因为在非widen2运算下：

第一个bf16对应被驱动到vfadd0里面的fp19模块，第二个bf16被驱动到fp32模块。但是在widen2运算条件下，bf16在vs1信号上自然排列，变成了第一个bf16应该被驱动到vfadd0的fp32模块上，第二个bf16应该被驱动到vfadd1的fp32模块上。

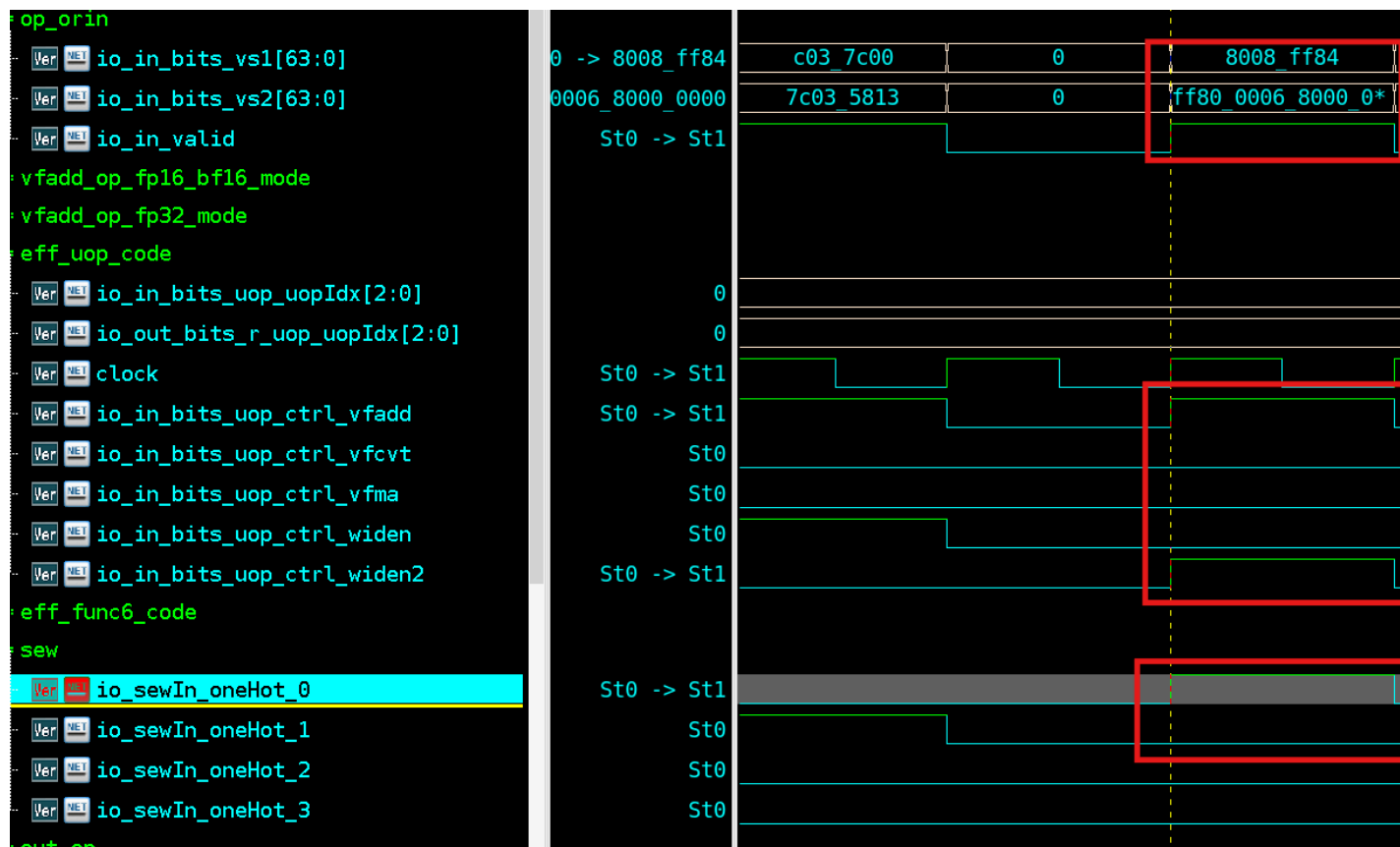
Q1: vfadd0/1模块的io_b信号，由于widen2指令的存在，不是一成不变的截取vs1的低32位/高32位，需要重新构造这一层选择逻辑。

Q2: 不能把subnorm_high变成low，应该引入widen2控制信号，重新构造选择逻辑，或者在Q1处把widen、widen2处的逻辑重新梳理到一个统一的点。

2、widen2指令下，NaN判断没有正确传递

BUG说明

- tc_name:test_tc2_widen
- tc_seed:0 在Widen2指令下，输入一组bf16数与一组FP32相加，DUT未正确传递NaN的情况
- 波形文件: DUTLaneFAdd_test_tc2_widen_faaed885.fst
- 日志文件: test_tc2_widen_0.log



根据Uop解析，此处为widen2类型的vfadd指令，应该将vs1低32位bit的两个bf16和vs2对应的两个FP32做运算。

根据解析，两组数如下：

VS1: 000000008008ff84

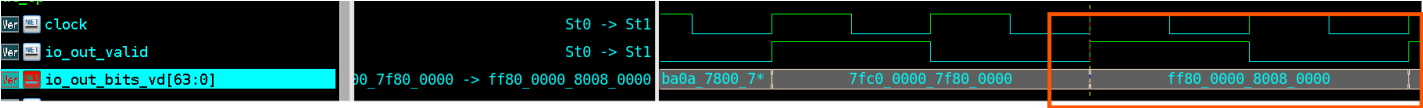
VS2: ff80000680000000

对应要做操作的两个数类型解析如下：

FP Pair 0:		
	OPA	OPB
Field	A	B
Precision	BF16	FP32
Category	NAN	ZERO
Sign	1	1
Exponent	0xff	0x0
Mantissa	0x4	0x0
Value Bits	0x0000ff84	0x80000000
Float Value	nan	-0.0

FP Pair 1:		
	OPA	OPB
Field	A	B
Precision	BF16	FP32
Category	SUBNORMAL	NAN
Sign	1	1
Exponent	0x0	0xff
Mantissa	0x8	0x6
Value Bits	0x00008008	0xff800006
Float Value	-7.34684e-40	nan

也就是操作类型中均有NaN出现，但是实际结果中未出现NaN



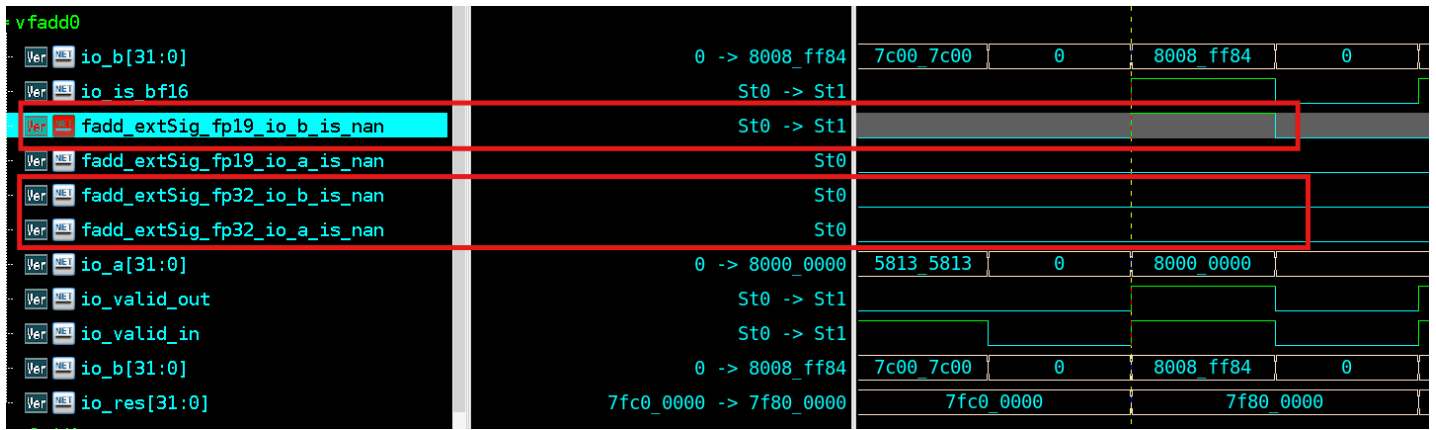
实际结果为ff80_0000，8008_0000，均不符合NaN。

定位分析

由于目前Scala生成的RTL可阅读性相对较差，此处分析不一定完全正确。

当BF16+FP32时，按照通路类型，应该是会复用vFADD内部的FP32加法器。

在第一组BF16+FP32相加是，trace到vfadd0时发现nan只在fp19这个加法器这里判断了，可能会没有正确传递到后面。



修改建议

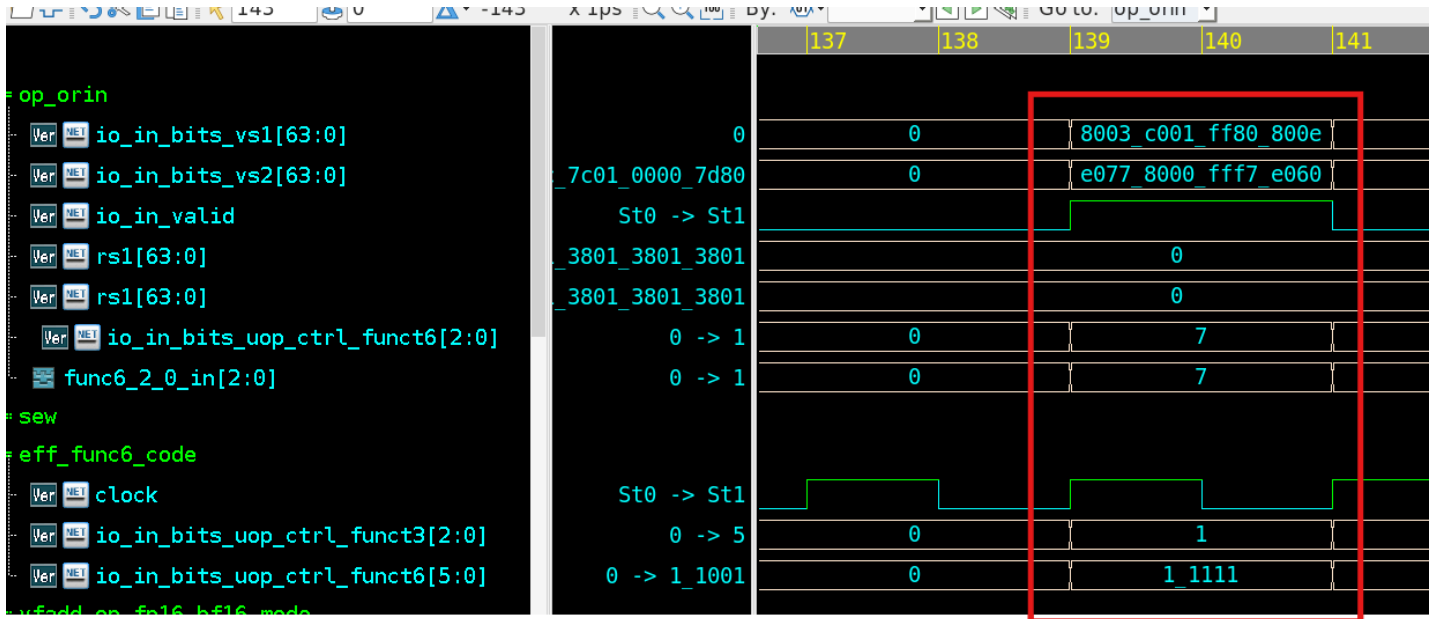
本质上和上面是一套问题吧，每个元素的附带信号并没有被驱动到正确的位置。

3、比较指令下，func6的控制信号和数据信号没有对齐，导致出现错误(1.8更新)

BUG说明：

- tc_name: test_cmp
- tc_seed: 0
- 波形文件: Report/test_cmp_0_report/DUTLaneFAdd_test_cmp_85bd94bb.fst
- log文件: Report/test_cmp_0_report/test_cmp_0.log

输入输出：



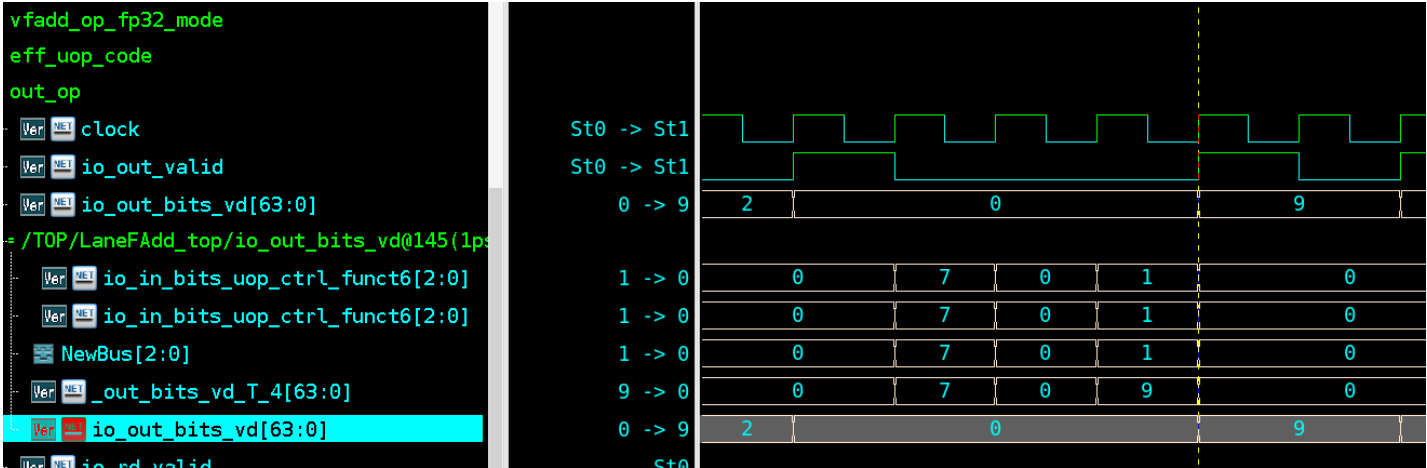
输入解析为下：

(详细见log文件line: 3781行)

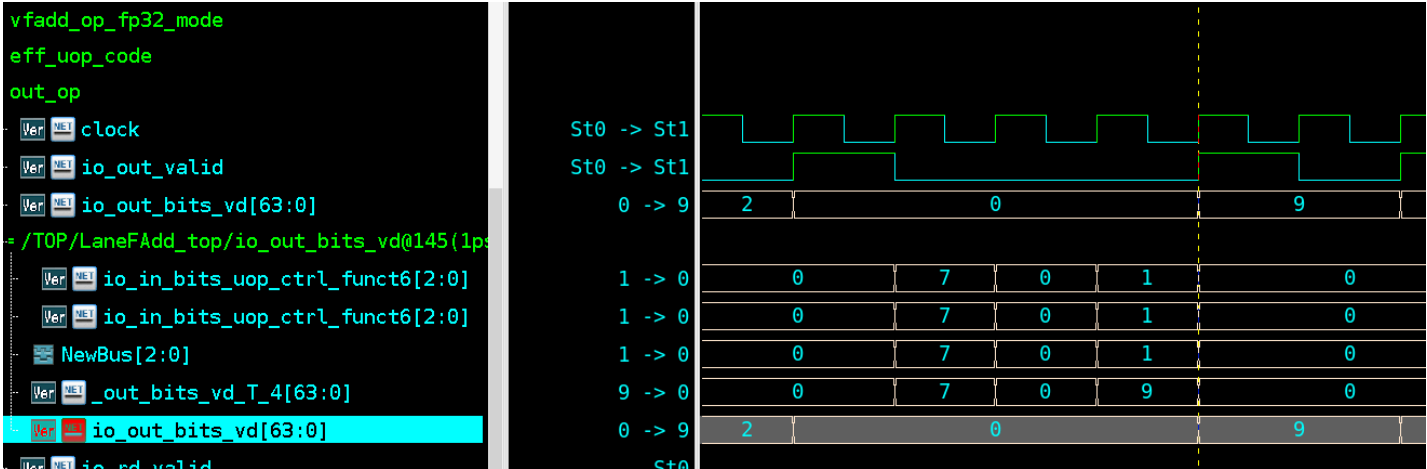
Field	Value
HL.Uop Type	VMFGE
HL.Mode	OPFVV
HL.SEW	BF16
HL.RD Idx	0
HL.Legal	0

Pair	Operand	Precision	Category	Sign	Exponent	Mantissa	Value Bits	Float Value
0	OPA	BF16	SUBNORMAL	1	0x0	0xe	0x0000800e	-1.2857e-39
0	OPB	BF16	NORMAL	1	0xc0	0x60	0x0000e060	-6.45636e+19
1	OPA	BF16	INFINITY	1	0xff	0x0	0x0000ff80	-inf
1	OPB	BF16	NAN	1	0xff	0x77	0x0000fff7	nan
2	OPA	BF16	NORMAL	1	0x80	0x1	0x0000c001	-2.01562
2	OPB	BF16	ZERO	1	0x0	0x0	0x00008000	-0
3	OPA	BF16	SUBNORMAL	1	0x0	0x3	0x00008003	-2.75506e-40
3	OPB	BF16	NORMAL	1	0xc0	0x77	0x0000e077	-7.11929e+19

根据指令格式可以得到，对应的比较结果是01X0，其中X表示有NaN,因此X也为0，



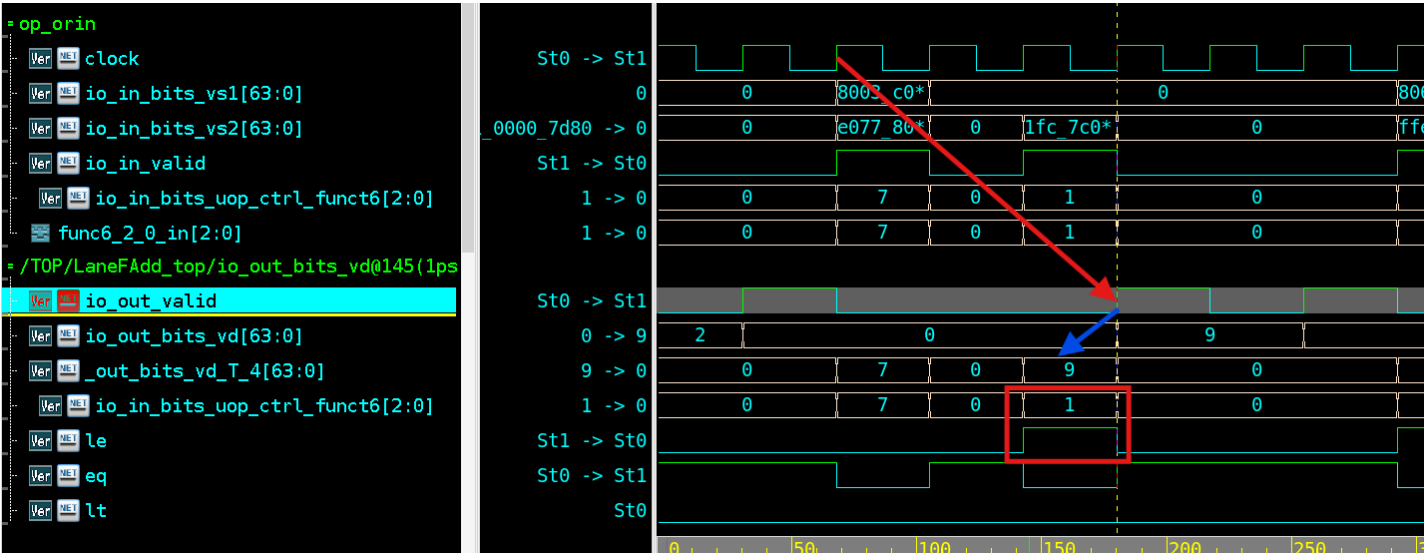
输出波形：



实际结果为0x09

定位分析

eq、le控制信号产生有误



```
wire eq = io_in_bits_uop_ctrl_funct6[2:0] == 3'h0;
wire le = io_in_bits_uop_ctrl_funct6[2:0] == 3'h1;
wire lt = io_in_bits_uop_ctrl_funct6[2:0] == 3'h3;
wire ne = io_in_bits_uop_ctrl_funct6[2:0] == 3'h4;
wire gt = io_in_bits_uop_ctrl_funct6[2:0] == 3'h5;
wire ge = io_in_bits_uop_ctrl_funct6[2:0] == 3'h7;
```

如波形图所示，Time 139输入后 Time 145输出，输入输出固定三个cycle（红色箭头）

蓝色箭头指向输出信号前一拍信号，trace发现，这个信号受eq、le、lt这些控制信号驱动。

但是这个控制信号是由当前输入的funct6产生的，实际上应该由之前采样的信号打拍来产生。导致判断此处执行的指令是le,而不是ge,上述提到，比较结果理论上是01x0，其中x表示有NaN,如果是执行le的话，除了x其他取反就是10x1，x有NaN，因此输出0，所以结果是1001,也就是0x09。

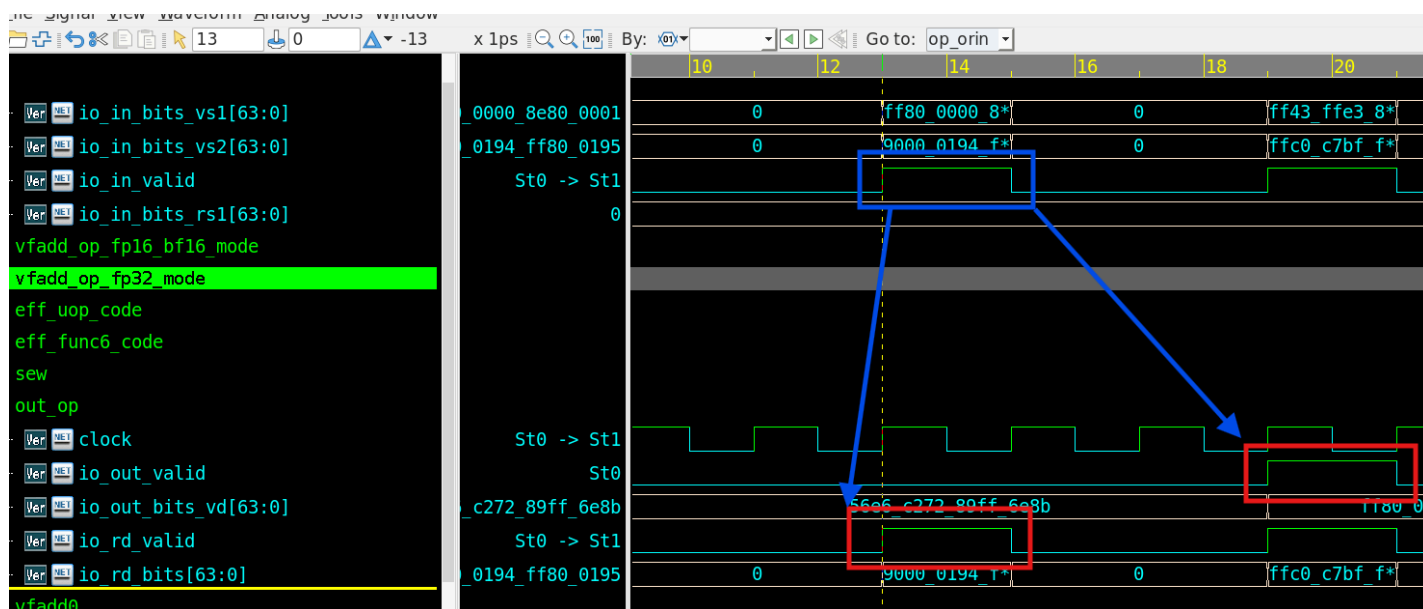
修改建议：

应该新增一条pipe，用来寄存器这些控制信号，并将line1751~1756处修改为寄存打拍后产生。

4、vfmv.f.s指令，io_out_rd_valid信号和io_out_valid信号不同时拉高。（1.11更新）

BUG说明：

- tc_name:test_vfmv_f_s
- tc_seed:0
- 波形文件:
- 日志文件:



vfmv.f.s指令会把vector寄存器的value mov到标量寄存器，输出是io_rd_valid,但是io_out_valid信号也会在收到指令后固定三个cycle后拉高。

如波形图所示，第一个rd_valid在io_in_valid拉高后里面拉高（这里设计是组合逻辑输出），而io_out_bits_vd对应的valid信号也会拉高。

修改建议：

从设计角度来说，如果是没有强保序的要求，这个io_out_valid在vfmv.f.s指令下就不应该拉高。

如果说需要保留io_out_valid这个信号的拉高，也就是说这个信号是优先级最高的有效信号，是否应该做成同拍？

这个地方本质上是一条指令同时引起了两条路径的valid信号拉高（导致我在monitor侧收集transaction的时候依靠valid信号多收到了我发包的数量）

5、多指令序列发生后的保序以及流水问题（1.11 更新）

从Bug4中发现，实际上该模块存在**立即能够执行完成的指令（vfmv.f.s）**，所以整个模块目前看起来对这个部分似乎没有考虑，也就是**单周期指令可能会超车前面指令的问题**。如果是不考虑顺序，也就是目标寄存器是scalar的和vector的是两条独立的路径，允许乱序（有可能上下游模块其实已经维护了相关性有ROB存在,那验证环境会对此做出一些修改

另外，vfmv.f.s这条指令的问题，既然这条指令都在第一个cycle完成了，为什么还要进入后级流水呢？这样做似乎对功耗不太友好

6、Widen情况下，NaN的判断逻辑有误（对Bug2的增补）

BUG说明

tc_seed: 0

tc_name:test_widen_random

波形路径: /wave_log/test_widen_random_0

log路径: /wave_log/test_widen_random_0

由于这个case还有其他错误，目前我trace的包对应log文件line 122行，Samplettime记录了发包对应的时间，方便DE trace

定位分析

```
723 .io_a_is_nan
724 (is_16 ? exp_is_all1s_2 & ~frac_is_0_16_2 : exp_is_all1s_2 & ~frac_is_0_32_0),
// home/long/work/riscv-vector-core/src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala:41:23, :70:75, :72:
725 in_b_is_nan
```

在widen2的情况下，723行对应应该按照FP32的方式判断NaN.

7、Widen2情况下，内部实现FP16向FP32的转换后，移位标准错误导致精度损失

BUG说明:

tc_seed: 0

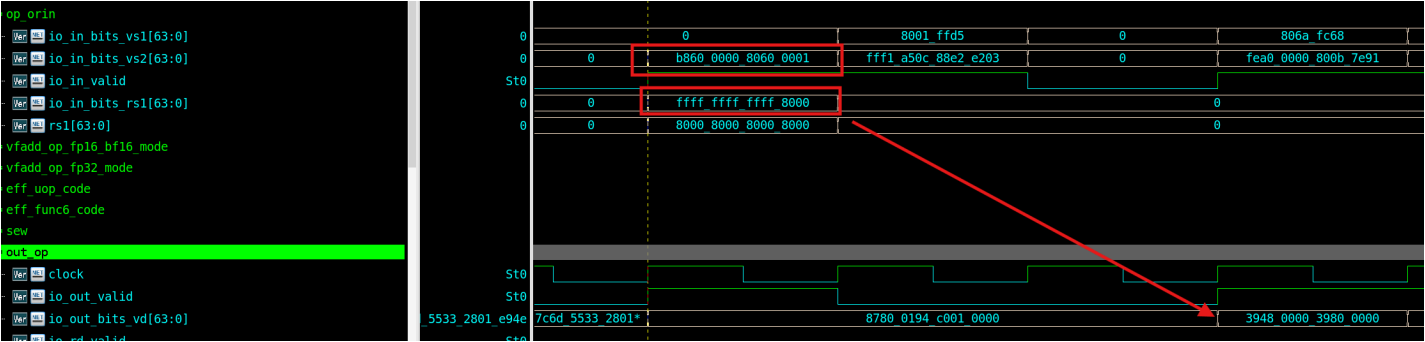
tc_name:test_widen_random

波形路径: /wave_log/test_widen_random_0

log路径: /wave_log/test_widen_random_0

tx_id: 1（我为每个transaction都设置了一个ID，方便追踪日志，当前日志对应line 27行）

波形:



如图所示，输入的对应该相加的浮点数组应该是：

Pair	Operand	Precision	Category	Sign	Exponent	Mantissa	Value Bits	Float Value
0	OPA	FP16	ZERO	1	0x0	0x0	0x00008000	-0.0
0	OPB	FP32	SUBNORMAL	1	0x0	0x600001	0x80600001	-8.816209e-39
1	OPA	FP16	ZERO	1	0x0	0x0	0x00008000	-0.0
1	OPB	FP32	NORMAL	1	0x70	0x600000	0xb8600000	-5.340576e-05

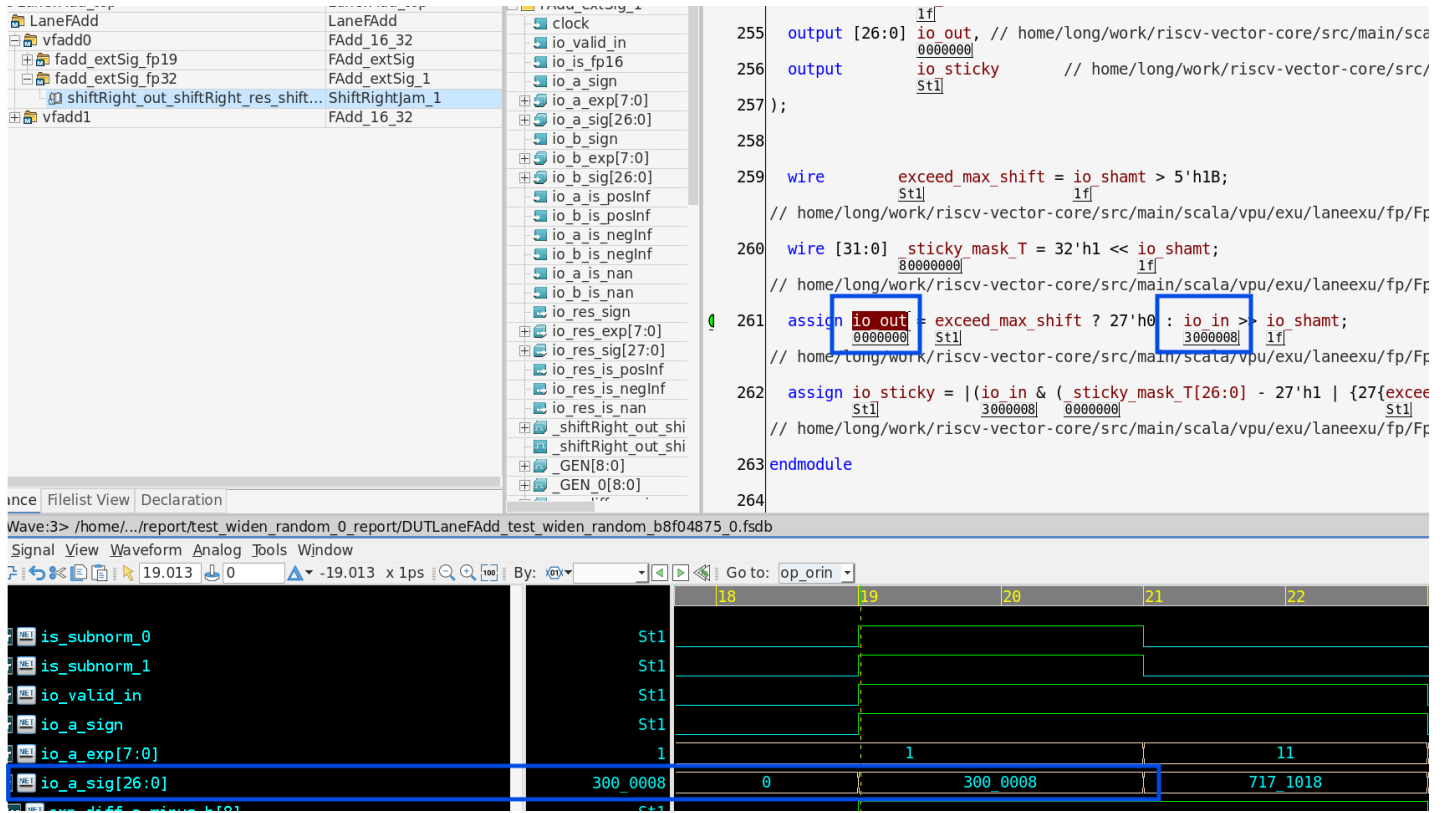
按照道理来说输出应该保持和vs2一致，因为都是加0减0。实际出现错误。

```
// home/long/work/riscv-vector-core/src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala
716 .io_b_exp
717 ((exp_is_0_3 ? 8'h1 : exp_high_b) + {1'h0, is_fp16_widen ? 7'h70 : 7'h0}),
// home/long/work/riscv-vector-core/src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala
```

line 717行，输入为0时，fp16的指数依然被暴力加了fp32和fp16之间的偏移差，会影响后面的对阶、移位。

对于widen1指令下而言，两个操作数的做了同样的操作。目前测试没有发生问题。

widen2指令下，fp16被的指数位修正为113(0x70+1)；本来是0的，导致fp32的尾数被移位。



深入说明

进一步分析可以发现，出现错误的本质并不是说进行对阶产生了错误，而是在有限位宽运算下，如果没有选择正确的对阶指数，会造成移位后bit精度损失，进一步导致错误。在FP运算的算法中，我们通常是采用大的指数作为对阶移位的标准，这样可以在有限移位器和保留位宽下最大限度保留精度(实际上在多操作数运算为了提高效率，也有选择中间指数或者统一对阶的实现)。目前这个case中，这种情况可以看到，选择了一个实际上小的exp作为移位对阶的标准，这样的话，虽然从数学真值上来说是等价的,举例来说：

下面表达式中，中间的表达式就是正常对阶的情况，最后一个表达式就是出现subnormal后可能出现的对阶情况，可以发现数学上是没有问题的，但核心是在移位的过程中，硬件运算一定是有限位宽的，所以会造成额外的精度损失，不符合我们的要求了。下面表达式中，如果我们限制小数位宽最多4位，可以发现最后一个表达式就损失了下划线后面的“11”。

$$1.1111 * 2^{10} + 0.0001 * 2^{12} = 2^{10}(1.1111 + 0.0100) = 2^{12}(0.0111_11 + 0.0001)$$

因此这种情况不单单是FP16输入为ZERO会出现，只要是subnormal就会出现（因为这个被转换的subnormal FP16 没有进行LZC真正的把指数对齐到隐藏1的位置）

补充Case

为了更好的说明这种情况，我重新跑了一个定向的case输入，方便DE查看

case:test_directed

约束如下：

```
with tx.randomize_with() as t:
    t.uop_type.inside(vsc.rangelist(UopType.VFWADD_W, UopType.VFWSUB_W))
```

```
t.sew_type == SewType.FP16
t.funct3_type == Funct3Type.OPFVW
t.is_legal == 1

with vsc.foreach(t.fp_pairs, idx=True) as i:
    t.fp_pairs[i].a.category == FpCategory.SUBNORMAL
    t.fp_pairs[i].b.category == FpCategory.NORMAL
    t.fp_pairs[i].b.exp.inside(vsc.rangelist(105, 104))

for pair in tx.fp_pairs:
    pair.b.man = 0x003

# 3. 重新调用 post_randomize 以更新 val_bits 和 payload
tx.post_randomize()
```

我把FP32的小数尾约束成0x003。核心是因为这样最低两位是1，所以移位更容易移出去导致错误。

FP32的指数尾设计成104和105。核心是因为subnormal转换成FP32后是113。这样可以保证是FP32向FP16转换后的数进行对阶。

这个case会发现大多数都是错误的，但也会有可能有一部分正确，原因是因为因为舍入实际上增加了额外的保留位，所以0x003移位后运气好的时候可能会不出错。

修改建议：

针对widen2指令，本质上FP16+FP32的混精度加法情况，转换规则需要进一步严格加强。

PS：这条case实际上非常有意思，刚好打出了zero+normal和zero+subnormal的情况，两个都出现了错误。我自己查看后面日志的时候，其实还发现了一种zero+normal的情况，**但是结果是对的！原因应该是normal的指数非常大！比zero转换后的0x71还大！**所有没有影响对阶（因为零怎么移位都是0），而上面所示的这个transaction运气非常差，刚好FP32的指数是0x70，他如果是0x71我们就发现不了这个问题了！

8、VFMIN MAX指令，在操作数有来自rs的情况下，选择输出错误。

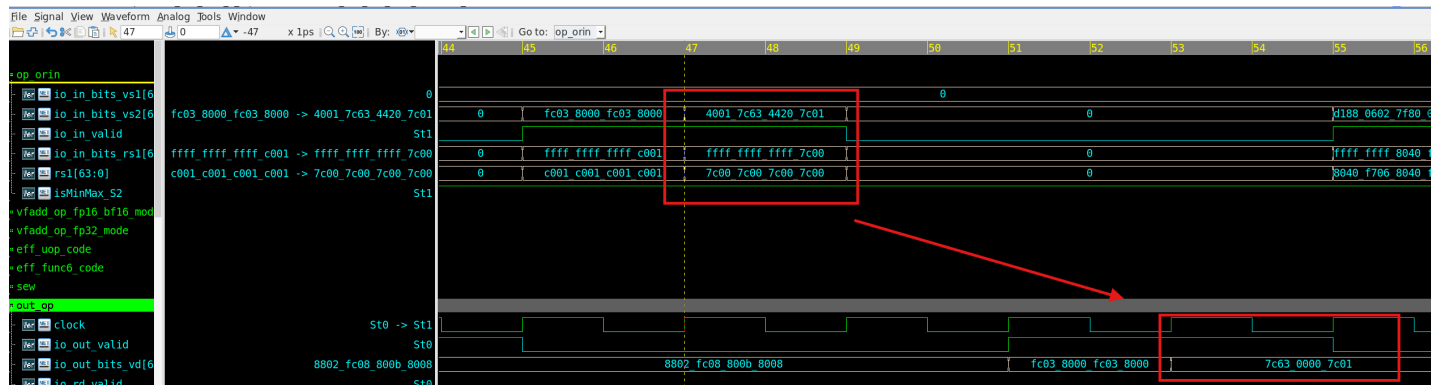
BUG说明：

tc_name:test_min_max

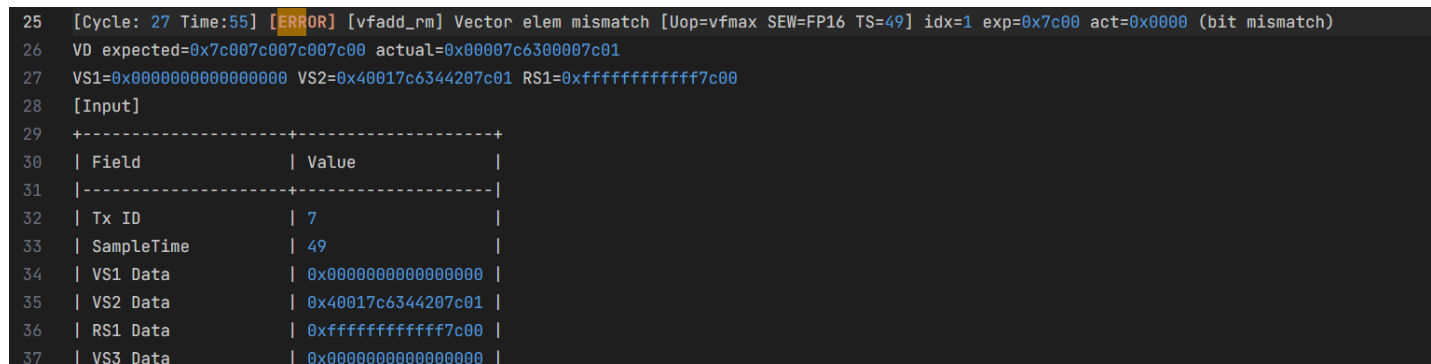
tc_seed:0

波形路径:test_min_max_0_report/*.vcd

日志路径:test_min_max_0_report/*.log

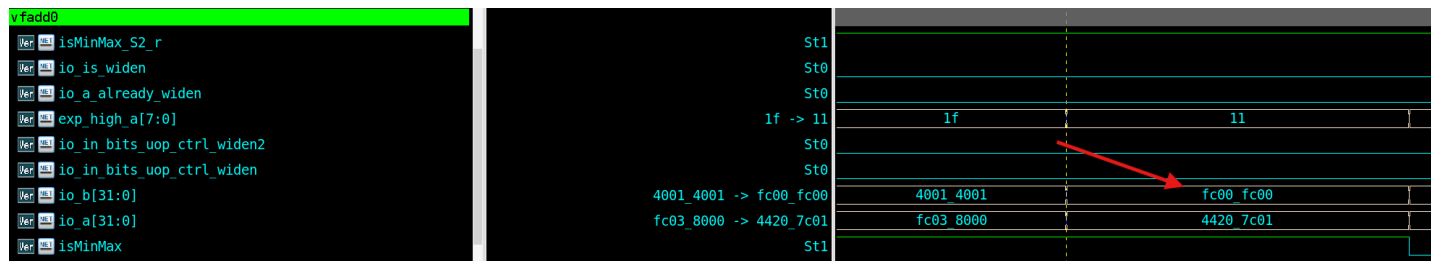


波形输入如上，当操作数来源为rs时，输出错误，7c00为正Inf，因此应该最大。



定位分析

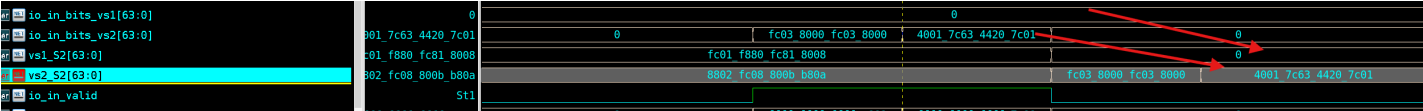
FMIN_MAX指令本质上做FSUB减法即可，trace波形发现，rs的操作数正确驱动到了FADD的端口，因此这部分逻辑没有问题



问题应该处在输出的选择逻辑，line 1434行显示，minmax选择逻辑是选择的vs1或者vs2

```
1433         isMinMax_S2
1434         ? {~(_vfadd1_io_res[31]) & isMax_S2 | _vfadd1_io_res[31] & isMin_S2
1435           ? vs2_S2[63:48]
1436           : vs1_S2[63:48],
1437         (res_is_16b_S2
1438           ? ~(_vfadd1_io_res[15]) & isMax_S2 | _vfadd1_io_res[15] & isMin_S2
1439           : ~(_vfadd1_io_res[31]) & isMax_S2 | _vfadd1_io_res[31] & isMin_S2)
1440         ? vs2_S2[47:32]
1441         : vs1_S2[47:32],
1442         ~(_vfadd0_io_res[31]) & isMax_S2 | _vfadd0_io_res[31] & isMin_S2
1443         ? vs2_S2[31:16]
1444         : vs1_S2[31:16],
1445         (res_is_16b_S2
1446           ? ~(_vfadd0_io_res[15]) & isMax_S2 | _vfadd0_io_res[15] & isMin_S2
1447           : ~(_vfadd0_io_res[31]) & isMax_S2 | _vfadd0_io_res[31] & isMin_S2)
1448         ? vs2_S2[15:0]
1449         : vs1_S2[15:0]}}
```

但是这个vs1_s2和vs2_s2根据波形发现是未经任何处理的vs1、vs2的打拍



修该建议

可以选择将rs寄存并修改line1433处的选择逻辑，或者在vs1打拍进入流水时增加选择逻辑(vs1或者extend之后的rs)，选择"真正进行操作的数据"进行打拍寄存

重测情况

问题清单

Bug ID	反馈日期	设计确认	设计反馈	重测情况	重测日期	回归需要	Fix	说明
Bug 1		已确认	已增补RTL	小范围Pass	1.12	需要	fix	待回归
Bug2		已确认		Fail		需要	not fix	-

Bug ID	反馈日期	设计确认	设计反馈	重测情况	重测日期	回归需要	Fix	说明
Bug3		已确认	已增补RTL	小范围Pass	1.13		not sure	VFCMP类指令回归仍有错误，不确定是否完全fix，目前可以直接运行cmp用例复现BUG
Bug4 & 5		已确认	已增补RTL	Pass	1.12	需要	fix	-
Bug6		已确认		Fail		需要	not fix	6本质上是2的增补，可以和2合并
Bug7	1.13	已确认				需要		
Bug8	1.13							

1.13 更新

目前Bug1、3均Rerun Pass

1.12 更新 BUG（1） rerun fail

重跑之前Bug1的case

log文件：

波形文件：

定位分析：

line 710行

```
708 .io_a_exp
709 ((exp_is_0_2 ? 8'h1 : exp_high_a)
710 + {1'h0, is_fp16_widen & ~io_a_already_widen ? 7'h70 : 7'h0}),
// home/long/work/riscv-vector-core/src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala:49:20, :69:65, :93:31, :97:{31,63,68,83,86}, :231:44
711 .io_a_sig
712 ({~exp_is_0_2,
713 is_16 & ~io_a_already_widen ? {frac_high_a_16, 13'h0} : io_a[22:0],
714 {st0->st1, st0->st1, 000->107, 000000->0719a4}
```

line 520行

```

3 wire [7:0] exp_high_a = io_is_fp16 ? {3'h0, io_a[30:26]} : io_a[30:23];
// home/long/work/riscv-vector-core/src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala:49:{20,34,52}

1 wire [7:0] exp_low_a = io_is_fp16 ? {3'h0, io_a[14:10]} : io_a[14:7];
// home/long/work/riscv-vector-core/src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala:50:{19,33,51}

2 wire [7:0] exp_high_b = io_is_fp16 ? {3'h0, io_b[30:26]} : io_b[30:23];
// home/long/work/riscv-vector-core/src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala:51:{20,34,52}

3 wire [7:0] exp_low_b = io_is_fp16 ? {3'h0, io_b[14:10]} : io_b[14:7];
// home/long/work/riscv-vector-core/src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala:52:{19,33,51}

4 wire [9:0] frac_high_a_16 = io_is_fp16 ? io_a[25:16] : {io_a[22:16], 3'h0};
// home/long/work/riscv-vector-core/src/main/scala/vpu/exu/laneexu/fp/fadd_16_32/FAdd_16_32.scala:56:{24,38,56,61}

5 wire [9:0] frac_low_a_16 = io_is_fp16 ? io_a[9:0] : {io_a[6:0], 3'h0};

```

此处对操作数a的exp的扩展情况有误，在widen2指令下，exp_high_a应该取原来的FP32的exp。因为此时io_a输入的是一个完整的FP32格式输入，也符合widen指令要求。

修改建议

```
line 520: wire [7:0] exp_high_a = io_is_fp16 & ~io_a_already_widen ? {3'h0, io_a[30:26]} : io_a[30:23];
```

此处应该引入a_already_widen信号控制。

1.12 更新BUG（4）（5），Rerun Pass

小范围rerun确认问题已fix

RTL更新情况

目前使用的姜老师反馈后的RTL，并且做了如下修改：

原代码 line 520

```

Line 520: wire [7:0] exp_high_a = io_is_fp16 ? {3'h0, io_a[30:26]} : io_a[30:23]; //原代码
Line 520: wire [7:0] exp_high_a = io_is_fp16 & ~io_a_already_widen ? {3'h0, io_a[30:26]} :
io_a[30:23]; //修改后代码，完全fix Bug1

```